COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	08/18/2026	**DATE SUBMITTED:**	08/31/2026
NAME:	Troy Jerron A. Bautista	**INSTRUCTOR:**	Sir Jimlord Quejado

ACTIVITY NO. 6.1 Searching Techniques

1. PROCEDURE OUTPUT

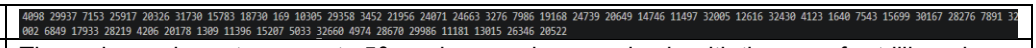
Screenshot	
Observations	The code used was to generate 50 random numbers randomly with the use of cstdlib and time.h STLs. Every time the numbers generated (denoted with i) is less than the max size, it will generate numbers infinitely until it reaches the max size of an array.

Table 6-1. Data Generated and Observations

Code	Linear Search Array:
	`template <typename T>`
	`void linearSearchArr(int arrSize, T dataFind, T list[]) {`
	`  // Step 1: I = 0`
	`  int i = 0;`
	`  // Repeat while I <= 0`
	`  while(i < arrSize){`
	`  // if (item = data[i])`
	`  if (dataFind == list[i]) {`
	`  // Print "Searching is successful"`
	`  std::cout <<"Searching is successful.\n";`
	`  return;`
	`  }`
	`  i++;`
	`  }`
	`  // Print "Searching is unsuccessful"`
	`  std::cout <<"Searching is unsuccessful.\n";`
	`}`
		Main Program:
		`int userInput;`
		`std::cout <<"Array Linear Search\n";`
		`std::cout <<"What number to search? ";`
		`std::cin >> userInput;`
		`std::cout <<"Array Result LS: ";`
		`linearSearchArr(max_size, userInput, dataset);`

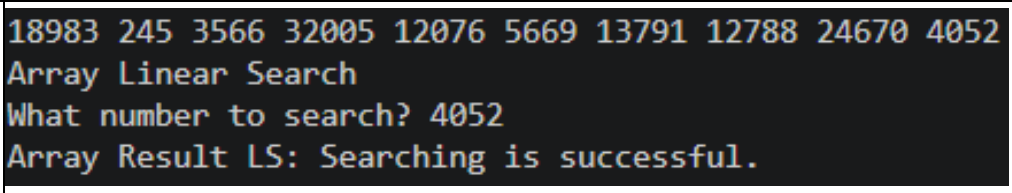
Screenshot	 18983 245 3566 32005 12076 5669 13791 12788 24670 4052 Array Linear Search What number to search? 4052 Array Result LS: Searching is successful.	
Observations	The code implements user input to determine if the number inputted is inside the array. First it will generate 10 random numbers and ask the user to input a number (7106), then it will cycle to the linear search algorithm and find if the number inputted is equal to every number in the list as long as i < arrSize is met. Since the algorithm found the number inputted, it will print that the search is successful.	

Table 6-2a. Linear Search for Arrays

Code	Linear Search Linked List: Main Prorgam:
Screenshot	
Observations	This code finds which node has the corresponding letter and outputs whether it is found or not. In the main program, new nodes are created for each individual character ('T', 'R', 'O', 'Y'), then in the linear search algorithm, while the head has not reached the end of the linked list, it will continue traversing until they find the key ('R'), if it is found, then the it outputs that the search was successful, but if not, it will point to the next node to check it again. If the key is not found, then it outputs that the searching is unsuccessful, like the result 2 in linked list search (finding 'r')

```
template <typename T>
void linearSearchLL(Node<T>* head, T dataFind) {
    while(head != nullptr) {
        if(head->data == dataFind) {
            std::cout <<"Searching is successful.\n";
            return;
        }
        head = head->next;
    }
    std::cout <<"Searching is unsuccessful.\n";
}
```

```
// create a linked list: T R O Y
Node<char>* name1 = new_node('T');
Node<char>* name2 = new_node('R');
Node<char>* name3 = new_node('O');
Node<char>* name4 = new_node('Y');

name1->next = name2;
name2->next = name3;
name3->next = name4;

std::cout <<"\nLinked List Linear Search\n";
std::cout <<"Linked List Result 1 LS: ";
linearSearchLL(name1, 'R');
std::cout <<"Linked List Result 2 LS: ";
linearSearchLL(name1, 'r');
```

```
Linked List Linear Search
Linked List Result 1 LS: Searching is successful.
Linked List Result 2 LS: Searching is unsuccessful.
```

Table 6-2b. Linear Search for Linked List

Code	Binary Search Array: Main Program:
Screenshot	
Observations	This program has taken a user input like in linear search array, but finds the middle in order to find the number key. After input the algorithm will use the formula to solve for the middle. Un fortunately, it is hard to visualize the output due to the numbers being unorganized as binary

```
template <typename T>
void binarySearchAR(int arrSize, T dataFind, T list[]) {
    // define low, high, and middle
    int low = 0, up = arrSize - 1, middle;
    // loop while low is less than or equal to high
    while (low <= up) {
        // compute for middle
        middle = int((low + up)/2);
        // if data found is middle -> data found -> return
        if (dataFind == list[middle]) {
            std::cout <<"The search element is found! \n";
            return;
        }
        // if data find < data middle -> new high
        else if (dataFind < list[middle]) {
            up = middle - 1;
        }
        else {
            // else update low = middle + 1
            low = middle + 1;
        }
    }
    // search is not found
    std::cout <<"The search element is not found! \n";
}
```

```
    std::cout <<"\nArray Binary Search \n";
    std::cout <<"What number to search? ";
    std::cin >> userInput;
    std::cout <<"Array Result BS: ";
    binarySearchAR(max_size, userInput, dataset);
```

```
18983 245 3566 32005 12076 5669 13791 12788 24670 4052
Array Binary Search
What number to search? 245
Array Result BS: The search element is found!
```

	search is recommended for organized linked list, but the implementation worked as it was able to find the number 245 in the binary array.	

Table 6-3a. Binary Search for Arrays

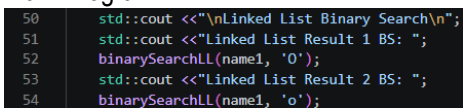
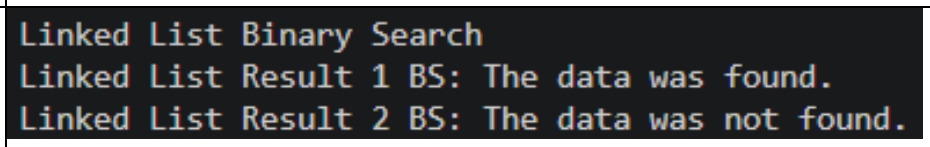
Code	Binary Search Linked List:  Main Program: 
Screenshot	
Observations	This code uses two functions order to successfully implement a binary search for linked list. Using the same linked list, it uses the getMiddle function to get the middle of the linked list. The getMIddle uses two parameters, the fast and slow function which are used to determine if the middle is more or less than the key. If the key is found, then it will output that the data has been found in the linked list, if not, it will determine either it's more or less than the key, and it will continue to loop as long as start does not point to last (nullptr). If the node reached the nullptr, then it will output that the data was not found.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

Node2.h

```
#ifndef NODE2_H
#define NODE2_H
#include <iostream>

class NodeTwo {
public:
    int data;
    NodeTwo *next;
};

NodeTwo *new_node (int newData) {
    NodeTwo *newNode = new NodeTwo;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

#endif
```

Array implementation:

```
template <typename T>
void LSearchArr (int ArrSize, T dataKey, T list[]) {
    int i = 0;
    int ArrCounter = 0;
    while (i < ArrSize) {
        if (dataKey == list[i]) {
            ArrCounter++;
            std::cout <<"Number Key is found! \n";
            std::cout <<"Number of comparisons: " <<ArrCounter <<std::endl;
            return;
        }
        i++;
        ArrCounter++;
    }
    std::cout <<"Number Key is not found! \n";
}
```

Linked List Implementation:

```
23  void LSearchLL (NodeTwo* head, int dataKey) {
24      int LLCounter = 0;
25      while (head != nullptr) {
26          if (head->data == dataKey) {
27              LLCounter++;
28              std::cout <<"Number is found! \n";
29              std::cout <<"Number of comparisons: " <<LLCounter <<std::endl;
30              return;
31          }
32          head = head->next;
33          LLCounter++;
34      }
35      std::cout <<"Number is not found! \n";
36  }
```

Main Program (Array):

```
 8  int main() {
 9      // Linear Search for Array
10      int Arr[size] = {5, 18, 2, 19, 18, 0, 8, 14, 19, 14};
11      std::cout <<"Finding 18 in the Array\n";
12      std::cout <<"Array Search Result for LS: ";
13      LSearchArr (size, 18, Arr);
```

Main Program (Linked List):

```
15      // Linear Search for Linked List
16      NodeTwo* Num1 = new_node(5);
17      NodeTwo* Num2 = new_node(18);
18      NodeTwo* Num3 = new_node(2);
19      NodeTwo* Num4 = new_node(19);
20      NodeTwo* Num5 = new_node(18);
21      NodeTwo* Num6 = new_node(0);
22      NodeTwo* Num7 = new_node(8);
23      NodeTwo* Num8 = new_node(14);
24      NodeTwo* Num9 = new_node(19);
25      NodeTwo* Num10 = new_node(14);
26
27      Num1->next = Num2;
28      Num2->next = Num3;
29      Num3->next = Num4;
30      Num4->next = Num5;
31      Num5->next = Num6;
32      Num6->next = Num7;
33      Num7->next = Num8;
34      Num8->next = Num9;
35      Num9->next = Num10;
36
37      std::cout <<"\nFinding 18 in the Linked List\n";
38      std::cout <<"Linked List Search Result for LS: ";
39      LSearchLL (Num1, 18);
```

Output:

```
Finding 18 in the Array
Array Search Result for LS: Number Key is found!
Number of comparisons: 2

Finding 18 in the Linked List
Linked List Search Result for LS: Number is found!
Number of comparisons: 2
```

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

Array Implementation:

```
6   template <typename T>
7   void InstanceArr (int ArrSize, T dataKey, T list[]) {
8       int i = 0;
9       int InstaArr = 0;
10      while (i < ArrSize) {
11          if (dataKey == list[i]) {
12              InstaArr++;
13          }
14          i++;
15      }
16      std::cout <<"Instance of " <<dataKey <<": " <<InstaArr <<std::endl;
17  }
```

Linked List Implementation:

```
19  void InstanceLL (NodeTwo* head, int dataKey) {
20      int InstaLL = 0;
21      while (head != nullptr) {
22          if (head->data == dataKey) {
23              InstaLL++;
24          }
25          head = head->next;
26      }
27      std::cout <<"Instance of " <<dataKey <<": " <<InstaLL <<std::endl;
28  }
```

Main Program (Array):

```
 9      int k;
10      // Linear Search for Array
11      int Arr[size] = {5, 18, 2, 19, 18, 0, 8, 14, 19, 14};
12      std::cout <<"Array Instance Count\n";
13      std::cout <<"Enter the number to count its occurance: ";
14      std::cin >> k;
15      InstanceArr (size, k, Arr);
```

Main Program (Linked List):

```
17      // Linear Search for Linked List
18      NodeTwo* Num1 = new_node(5);
19      NodeTwo* Num2 = new_node(18);
20      NodeTwo* Num3 = new_node(2);
21      NodeTwo* Num4 = new_node(19);
22      NodeTwo* Num5 = new_node(18);
23      NodeTwo* Num6 = new_node(0);
24      NodeTwo* Num7 = new_node(8);
25      NodeTwo* Num8 = new_node(14);
26      NodeTwo* Num9 = new_node(19);
27      NodeTwo* Num10 = new_node(14);
28
29      Num1->next = Num2;
30      Num2->next = Num3;
31      Num3->next = Num4;
32      Num4->next = Num5;
33      Num5->next = Num6;
34      Num6->next = Num7;
35      Num7->next = Num8;
36      Num8->next = Num9;
37      Num9->next = Num10;
38
39      std::cout <<"\nLinked List Instance Count\n";
40      std::cout <<"Enter the number to count its occurance: ";
41      std::cin >> k;
42      InstanceLL (Num1, k);
```

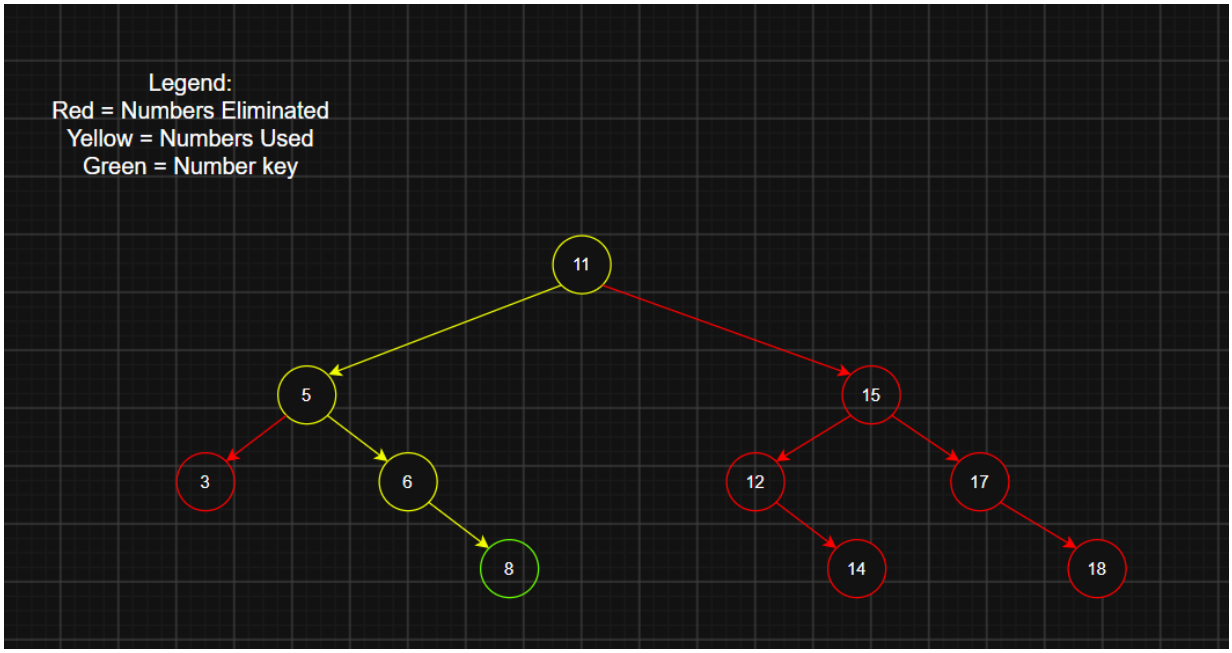
Output:

```
Array Instance Count
Enter the number to count its occurance: 19
Instance of 19: 2

Linked List Instance Count
Enter the number to count its occurance: 8
Instance of 8: 1
```

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Diagram:



Array Implementation:

```
43  template <typename T>
44  void BSearchArr (int ArrSize, T dataKey, T list[]) {
45      int low = 0, high = ArrSize - 1, middle, CompCountArr = 0;
46      while (low <= high) {
47          middle = int((low + high) / 2);
48          if (dataKey == list[middle]) {
49              CompCountArr++;
50              std::cout <<"\nGoing to: " <<list[middle] <<std::endl;
51              std::cout <<"This is the number key!\n";
52              std::cout <<"The number key has been found!\n";
53              std::cout <<"\nNumber of comparisons: " <<CompCountArr <<std::endl;
54              return;
55          }
56          else if (dataKey < list[middle]) {
57              CompCountArr++;
58              std::cout <<"\nGoing to: " <<list[middle] <<std::endl;
59              std::cout <<"This is not the number key!\n";
60              std::cout <<"Moving to next index...\n";
61              high = middle - 1;
62          }
63          else {
64              CompCountArr++;
65              std::cout <<"\nGoing to: " <<list[middle] <<std::endl;
66              std::cout <<"This is not the number key!\n";
67              std::cout <<"Moving to next index...\n";
68              low = middle + 1;
69          }
70      }
71      std::cout <<"The number key has not been found!\n";
72  }
```

Main Program (Array):

```
44      int NewArr[size] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
45      std::cout <<"\nFinding 8 in an array in BS\n";
46      BSearchArr(size, 8, NewArr);
```

Output:

```
Linked List Instance Count
Enter the number to count its occurance: 19
Instance of 19: 2

Finding 8 in an array in BS

Going to: 11
This is not the number key!
Moving to next index...

Going to: 5
This is not the number key!
Moving to next index...

Going to: 6
This is not the number key!
Moving to next index...

Going to: 8
This is the number key!
The number key has been found!

Number of comparisons: 4
```

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

Array Implementation:

```
30  template <typename T>
31  void BSearchArr (int ArrSize, int low, int high, T dataKey, T list[], int CompCountArr) {
32      int middle;
33      while (low <= high) {
34          middle = int((low + high) / 2);
35          if (dataKey == list[middle]) {
36              CompCountArr++;
37              std::cout <<"\nGoing to: " <<list[middle] <<std::endl;
38              std::cout <<"This is the number key!\n";
39              std::cout <<"The number key has been found!\n";
40              std::cout <<"\nNumber of comparisons: " <<CompCountArr <<std::endl;
41              return;
42          }
43          else if (dataKey < list[middle]) {
44              CompCountArr++;
45              std::cout <<"\nGoing to: " <<list[middle] <<std::endl;
46              std::cout <<"This is not the number key!\n";
47              std::cout <<"Moving to next index...\n";
48              return BSearchArr(ArrSize, low, middle - 1, dataKey, list, CompCountArr);
49          }
50          else {
51              CompCountArr++;
52              std::cout <<"\nGoing to: " <<list[middle] <<std::endl;
53              std::cout <<"This is not the number key!\n";
54              std::cout <<"Moving to next index...\n";
55              return BSearchArr(ArrSize, middle + 1, high, dataKey, list, CompCountArr);
56          }
57      }
58      std::cout <<"The number key has not been found!\n";
59  }
```

Main Program (Array):

```
43      int NewArr[size] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
44      std::cout <<"\nFinding 8 in an array in BS\n";
45      BSearchArr(size, 0, size - 1, 8, NewArr, 0);
```

Output:

```
Finding 8 in an array in BS

Going to: 11
This is not the number key!
Moving to next index...

Going to: 5
This is not the number key!
Moving to next index...

Going to: 6
This is not the number key!
Moving to next index...

Going to: 8
This is the number key!
The number key has been found!

Number of comparisons: 4
```

Conclusion:

In conclusion, searching techniques are ways to find characters and numbers inside a list. They can either be a linear or binary searching algorithm. Linear searching algorithm searches the data by checking the key if it's inside the list from beginning to end, while binary searching algorithm finds the middle determines if that middle is the key, more than the key or less than the key.

The procedures show how to implement searching algorithm while having different kinds of list to find the key. The array implementation uses integers with a fixed size using randomized number to find the key, while linked list uses characters to find the key. The output shows how finding both number in the array worked, while the linked list showed that it is case-sensitive since it does find 'R and 'O', but not 'r' and 'o'.

The supplementary activity was similar to the procedure, but linked list now uses integer, needs the count for comparisons, and the implement instances of each number in the list and recursive binary search algorithm. Problem 1 is simple as you need to have a new counter parameter that increment every time it is found or not. Problem 2 is also similar to problem 1 as it you still implement the counter when the number is found, but remove the return function. Problem 3 was also similar to problem 1, and problem 4 requires you to use the return function to return back to the function until the number was found.

Overall the assignment was not too nard, since most of the supplementary problems can easily be solved by using the code of the procedures, which contributed to the easy implementation of both array and linked list. I would want to improve more on making recursive search algorithm, since I did have problems getting problem 4 to work with

errors, or problems if the code executed.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of activity and grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Using ChatGPT to clarify the instructions of the supplementary activity without any help from making the program**
Using Grammarly to check for corrections and polish the conclusion

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.